Designing Microservices & Data Management

Characteristics of Well-Designed Microservices

Single Responsibility Principle (SRP): Each microservice should focus on a single capability.

Independence: Services should be independently deployable and testable.

Decentralized Data Management: Each microservice owns its data.

Resilience: Failures in one service should not crash the entire system.

API-First Communication: Services interact via APIs (REST, gRPC, GraphQL, Messaging Queues).

Designing Microservices: Key Considerations

Service Boundaries

- **Domain-Driven Design (DDD):** Identify business domains and subdomains.
- **Bounded Contexts:** Group related functionality (e.g., Order Service, Payment Service).

Designing Microservices: Key Considerations

Communication Patterns

- **Synchronous (Request/Response):** REST, gRPC.
- **Asynchronous (Event-driven):** Message brokers like Kafka, RabbitMQ.
- **Best Practice:** Prefer async messaging to reduce tight coupling.

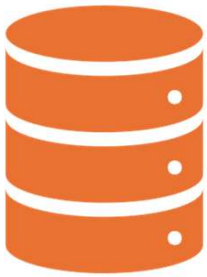
Designing Microservices: Key Considerations

Deployment & Scaling

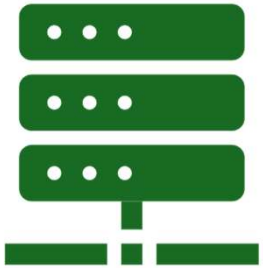
- **Containerization:** Docker + Kubernetes.
- **Auto-scaling:** Services scale based on load independently.
- **Service Discovery:** Tools like Consul, Eureka, or Kubernetes services.

Data Management in Microservices

The Challenge

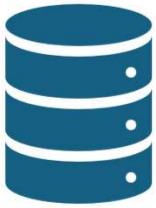


In monolithic systems → single database for all modules.



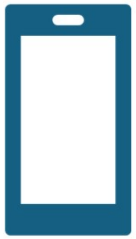
In microservices → database per service (ensures independence but introduces complexity).

Database per Service Pattern



Each microservice **owns its schema and database**.

Services never directly query another service's database.



Communication happens via APIs or messaging.

Pros VS Cons

Pros:	Autonomy & scalability	No single point of failure at the data layer
Cons:	Data duplication	Complexity in ensuring consistency

Data Consistency Approaches

Strong Consistency: Similar to monoliths but harder to scale.

Eventual Consistency: Common in microservices. Data updates propagate asynchronously.

Data Management Techniques

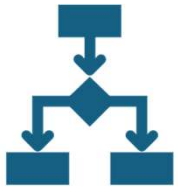
Sagas Pattern: Long-running transactions split across multiple services. Coordination via:

- Choreography (event-based)
- Orchestration (central controller)

CQRS (Command Query Responsibility Segregation): Separate read and write models for efficiency.

Event Sourcing: Store changes as a sequence of events rather than current state.

Putting it all together



- **Sagas:** Handle distributed transactions reliably.
- **CQRS:** Optimize performance by splitting read/write responsibilities.
- **Event Sourcing:** Provide an auditable history of data changes.

These patterns are often **combined**:

- Use **Event Sourcing + CQRS** to manage data storage.
- Use **Sagas** to coordinate long-running business processes.

Visual Summary

```
+-----------------------------------------------+
|                Data Mgmt in Microservices     |
+-----------------------------------------------+
| Sagas          | CQRS              | Event Sourcing|
|----------------/-------------------/----------------/
| Distributed Tx | Separate R/W      | Store events   |
| Choreography   | Command model     | Replay to state|
| Orchestration  | Query model       | Immutable log  |
+-----------------------------------------------+
```

Data Sharing Strategies

API Composition: Aggregate data from multiple services via APIs.

Database View Replication: Create read-only views for reporting.

Event-Driven Data Replication: Services publish domain events, and other services update their local data stores.

Questions

In an online shopping system, when would **choreography** be better than **orchestration**?

Why might CQRS improve performance in a system with **millions of daily reads but fewer writes**?

How does Event Sourcing help with **auditing and debugging**?

Data Storage Choices in Microservices

Polyglot Persistence

Choose the best database per microservice (SQL, NoSQL, Graph DB).

Examples:

- Order Service → Relational DB (MySQL, PostgreSQL)
- Product Catalog → NoSQL (MongoDB, DynamoDB)
- Search Service → ElasticSearch
- Analytics Service → Data Warehouse (Snowflake, BigQuery)

Security & Governance

Security & Governance

Authentication & Authorization: Centralized Identity Provider (e.g., OAuth2, OpenID Connect).

Data Security: Encrypt sensitive data at rest and in transit.

Governance: API Gateway for access control, monitoring, throttling.

Observability & Monitoring

- **Logging:** Centralized logging (ELK Stack, Splunk).
- **Metrics:** Prometheus, Grafana for service health.
- **Tracing:** Distributed tracing with Jaeger or Zipkin.